

Bloque 1 — Subdivisión en 3 Sub-Issues

Contexto

El Bloque 1 implementa el servicio de extracción de texto PDF con pypdf. Se divide en 3 sub-issues independientes que pueden trabajarse en paralelo, sincronizándose únicamente en los contratos de interfaz definidos aquí.

Stack: Python 3.12+ · pypdf · pydantic-settings · pytest · uv

Mapa de Dependencias

[!] IMPORTANTE

El 1A debe completarse primero (o al menos definir los contratos) porque 1B y 1C dependen de ExtractedDocument y Settings. En la práctica, se puede mergear 1A en ~1 hora y liberar 1B y 1C en paralelo.

Sub-issue 1A — Configuración y Modelos de Datos

Issue: feat: definir modelos de datos y configuración por entorno (Bloque 1A)

Responsable: Integrante A

Tiempo estimado: 1–2 horas

Prioridad: BLOQUEANTE para 1B y 1C — debe mergearse primero

Objetivo

Definir los contratos de datos que utilizarán los demás sub-issues: el modelo ExtractedDocument y la configuración centralizada Settings.

Archivos a crear/modificar

[NEW] `app/services/extractor/models.py`

```

"""
Modelos de datos del servicio de extracción PDF.
Contrato compartido entre PDFExtractorService y la capa de API/DB.
"""

from dataclasses import dataclass, field

@dataclass
class ExtractedDocument:
    """
    Representa el resultado de extraer texto de un PDF.

    Atributos:
        text (str): Texto extraído de todas las páginas del PDF.
        checksum (str): Hash SHA-256 del archivo original (hex digest).
        page_count (int): Número total de páginas del PDF.
        metadata (dict): Metadatos del PDF (autor, título, fecha, etc.).
    """
    text: str
    checksum: str
    page_count: int
    metadata: dict = field(default_factory=dict)

```

[NEW] `app/core/config/settings.py`

```

"""
Configuración centralizada usando pydantic-settings (12-Factor III).
Lee variables de entorno y del archivo .env.
"""

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """Configuración de la aplicación leída desde variables de entorno."""

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
        extra="ignore",
    )

    # Aplicación
    APP_NAME: str = "pdf-extracttext"
    APP_ENV: str = "development"
    LOG_LEVEL: str = "INFO"

    # MongoDB (usado por Bloque 2)
    MONGODB_URI: str = "mongodb://localhost:27017"
    MONGODB_DB_NAME: str = "pdf_extracttext"

    # Validación de PDF
    MAX_FILE_SIZE_MB: int = 10
    PDF_ALLOWED_MIME_TYPE: str = "application/pdf"

# Singleton – importar desde aquí en toda la app
settings = Settings()

```

[MODIFY] app/services/extractor/__init__.py

```

# extractor package
# Responsable de la extracción de texto desde archivos PDF.

from app.services.extractor.models import ExtractedDocument

__all__ = ["ExtractedDocument"]

```

[MODIFY] app/core/config/__init__.py

```

# config package
from app.core.config.settings import Settings, settings

__all__ = ["Settings", "settings"]

```

[NEW] .env.example

```
# =====
# pdf-extracttext – Variables de Entorno (12-Factor: Config)
# =====
# Copia este archivo como .env y completa los valores reales.
# NUNCA subas .env al repositorio.

# — Aplicación —
APP_NAME=pdf-extracttext
APP_ENV=development      # development | production | testing
LOG_LEVEL=INFO

# — Base de Datos (MongoDB) —
MONGODB_URI=mongodb://localhost:27017
MONGODB_DB_NAME=pdf_extracttext

# — Validación de PDF —
MAX_FILE_SIZE_MB=10
PDF_ALLOWED_MIME_TYPE=application/pdf
```

[MODIFY] requirements.txt

```
pypdf>=4.0.0
pydantic-settings>=2.0.0
python-dotenv>=1.0.0
fastapi>=0.110.0
uvicorn[standard]>=0.29.0
motor>=3.4.0
pytest>=8.0.0
pytest-asyncio>=0.23.0
httpx>=0.27.0
```

Criterios de aceptación

- ☐ ExtractedDocument es importable desde app.services.extractor
- ☐ settings es importable desde app.core.config
- ☐ settings.MAX_FILE_SIZE_MB retorna el valor del .env o el default 10
- ☐ .env.example cubre todas las variables usadas en Settings
- ☐ requirements.txt contiene todas las dependencias del proyecto

Principios

Principio	Aplicación
SRP	models.py solo define datos, settings.py solo define config
12-Factor III	Config exclusivamente por variables de entorno
KISS	Dataclass simple, sin lógica de negocio

Sub-issue 1B — Implementación de PDFExtractorService

Issue: feat: implementar PDFExtractorService con pypdf (Bloque 1B)

Responsable: Integrante B

Tiempo estimado: 2–3 horas

Requiere: Sub-issue 1A mergeado (o rama local con models.py y settings.py)

Objetivo

Implementar la clase PDFExtractorService que encapsula toda la lógica de extracción, validación y generación de checksum del PDF.

Archivos a crear/modificar

[NEW] `app/services/extractor/pdf_extractor.py`

```

"""
Servicio de extracción de texto desde archivos PDF.

Principio SRP: esta clase SOLO se encarga de procesar PDFs.
No persiste datos, no genera resúmenes ni interactúa con la API.
"""

import hashlib
import io
import logging

import pypdf
from pypdf import PdfReader
from pypdf.errors import PdfReadError

from app.services.extractor.models import ExtractedDocument

logger = logging.getLogger(__name__)

class PDFValidationError(ValueError):
    """Se lanza cuando el archivo no es un PDF válido o supera el tamaño máximo."""

class PDFExtractorService:
    """
    Servicio responsable de:
    1. Validar el PDF (formato y tamaño).
    2. Extraer texto de todas las páginas.
    3. Extraer metadatos del PDF.
    4. Calcular el checksum SHA-256.

    Uso:
        service = PDFExtractorService(max_file_size_mb=10)
        doc = service.extract(file_bytes)
    """

    def __init__(self, max_file_size_mb: int = 10) -> None:
        self._max_bytes = max_file_size_mb * 1024 * 1024

    # — Método principal —————

    def extract(self, file_bytes: bytes) -> ExtractedDocument:
        """
        Punto de entrada principal. Valida el PDF y retorna un ExtractedDocument.

        Args:
            file_bytes: Contenido del PDF como bytes (sin escritura en disco).

        Returns:
            ExtractedDocument con texto, checksum, page_count y metadata.

        Raises:
            PDFValidationError: Si el archivo no es PDF válido o es demasiado grande.
        """
        self.validate_pdf(file_bytes)
        reader = self._get_reader(file_bytes)

        return ExtractedDocument(
            text=self.extract_text(file_bytes),
            checksum=self.calculate_checksum(file_bytes),

```

```

        page_count=len(reader.pages),
        metadata=self.extract_metadata(file_bytes),
    )

# — Métodos públicos (también usados en tests unitarios) —

def validate_pdf(self, file_bytes: bytes) -> bool:
    """
    Valida que los bytes correspondan a un PDF válido y dentro del límite de tamaño.

    Args:
        file_bytes: Contenido del archivo.

    Returns:
        True si el archivo es válido.

    Raises:
        PDFValidationError: Si el archivo no es válido.
    """
    if len(file_bytes) > self._max_bytes:
        raise PDFValidationError(
            f"El archivo supera el tamaño máximo de {self._max_bytes // (1024 * 1024)} MB."
        )
    if not file_bytes.startswith(b"%PDF"):
        raise PDFValidationError("El archivo no tiene la firma PDF válida (%PDF).")
    try:
        self._get_reader(file_bytes)
    except PdfReadError as exc:
        raise PDFValidationError(f"El archivo PDF está corrupto o no es válido: {exc}") from exc
    return True

def calculate_checksum(self, file_bytes: bytes) -> str:
    """
    Calcula el checksum SHA-256 del archivo.

    Args:
        file_bytes: Contenido del archivo.

    Returns:
        Hash SHA-256 en formato hexadecimal (64 caracteres).
    """
    return hashlib.sha256(file_bytes).hexdigest()

def extract_text(self, file_bytes: bytes) -> str:
    """
    Extrae el texto de todas las páginas del PDF.

    Args:
        file_bytes: Contenido del PDF como bytes.

    Returns:
        Texto concatenado de todas las páginas, separado por saltos de línea.
        Retorna string vacío si el PDF no contiene texto seleccionable.
    """
    reader = self._get_reader(file_bytes)
    pages_text = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        pages_text.append(page_text)
    return "\n".join(pages_text).strip()

```

```

def extract_metadata(self, file_bytes: bytes) -> dict:
    """
    Extrae los metadatos del PDF (autor, título, fecha de creación, etc.).

    Args:
        file_bytes: Contenido del PDF como bytes.

    Returns:
        Diccionario con los metadatos disponibles. Claves con prefijo '/'
        de pypdf son normalizadas (e.g. '/Author' → 'author').
    """
    reader = self._get_reader(file_bytes)
    raw_meta = reader.metadata or {}
    return {
        key.lstrip("/").lower(): str(value)
        for key, value in raw_meta.items()
        if value is not None
    }

# — Método privado —————

def _get_reader(self, file_bytes: bytes) -> PdfReader:
    """Crea un PdfReader en memoria (sin escritura en disco)."""
    return PdfReader(io.BytesIO(file_bytes))

```


[MODIFY] app/services/extractor/_init_.py

```
# extractor package
# Responsable de la extracción de texto desde archivos PDF.

from app.services.extractor.models import ExtractedDocument
from app.services.extractor.pdf_extractor import PDFExtractorService, PDFValidationError

__all__ = ["ExtractedDocument", "PDFExtractorService", "PDFValidationError"]
```

Criterios de aceptación

- [] PDFExtractorService y PDFValidationError son importables desde app.services.extractor
- [] extract() no escribe ningún archivo en disco (usa io.BytesIO)
- [] validate_pdf() lanza PDFValidationError si el archivo no empieza con %PDF
- [] validate_pdf() lanza PDFValidationError si supera el tamaño máximo
- [] calculate_checksum() retorna exactamente 64 caracteres hex
- [] extract_text() retorna "" para PDFs sin texto seleccionable
- [] extract_metadata() retorna un dict con claves en minúsculas sin /
- [] Todos los métodos operan exclusivamente sobre bytes (sin rutas de archivo)

Principios

Principio	Aplicación
SRP	El servicio solo extrae; no persiste, no hace HTTP
DRY	Checksum centralizado en un único método reutilizable
KISS	Interfaz extract(bytes) → ExtractedDocument
YAGNI	No se implementa OCR ni resumen IA (otro bloque)

Sub-issue 1C — Tests TDD del Servicio de Extracción**Issue: test: escribir tests TDD para PDFExtractorService (Bloque 1C)**

Responsable: Integrante C

Tiempo estimado: 2–3 horas

Requiere: Sub-issue 1A mergeado; puede avanzar con 1B en paralelo usando mocks

Objetivo

Escribir la suite completa de tests unitarios para PDFExtractorService, siguiendo TDD: los tests documentan el comportamiento esperado y validan la implementación del sub-issue 1B.

Archivos a crear/modificar**[NEW] tests/unit/extractor/conftest.py**

```

"""
Fixtures de pytest para los tests del PDFExtractorService.
Los PDFs de prueba se generan en memoria con pypdf (sin archivos en disco).
"""

import io
import pytest
import pypdf
from pypdf import PdfWriter

def _build_pdf(text: str = "", author: str = "", title: str = "") -> bytes:
    """Helper interno: crea un PDF en memoria con pypdf."""
    writer = PdfWriter()
    page = writer.add_blank_page(width=595, height=842) # A4
    if text:
        # pypdf no tiene API de inserción de texto en PdfWriter;
        # usamos un PDF mínimo con texto embebido para pruebas reales.
        pass
    if author:
        writer.add_metadata({" /Author": author, " /Title": title})
    buf = io.BytesIO()
    writer.write(buf)
    return buf.getvalue()

@pytest.fixture
def sample_pdf_bytes() -> bytes:
    """PDF válido con al menos una página. Para tests de validación y checksum."""
    return _build_pdf(author="Test Author", title="Test PDF")

@pytest.fixture
def empty_pdf_bytes() -> bytes:
    """PDF válido sin texto seleccionable (página en blanco)."""
    return _build_pdf()

@pytest.fixture
def invalid_file_bytes() -> bytes:
    """Bytes que NO son un PDF válido."""
    return b"This is not a PDF file content at all."

@pytest.fixture
def oversized_pdf_bytes(sample_pdf_bytes: bytes) -> bytes:
    """PDF válido que supera el límite de 1 MB (para test de tamaño)."""
    # Padding para forzar superación del límite en tests con max=1
    return sample_pdf_bytes + b"\x00" * (1 * 1024 * 1024 + 1)

```

[NEW] tests/unit/extractor/test_pdf_extractor.py

```

"""
Tests unitarios para PDFExtractorService – Bloque 1C.

Sigue TDD: cada test documenta el comportamiento esperado
antes o en paralelo a la implementación (sub-issue 1B).
"""

import pytest

from app.services.extractor import PDFExtractorService, PDFValidationError, ExtractedDocument

# — Fixtures locales —————

@pytest.fixture
def service() -> PDFExtractorService:
    """Instancia del servicio con límite de tamaño reducido para tests."""
    return PDFExtractorService(max_file_size_mb=10)

@pytest.fixture
def strict_service() -> PDFExtractorService:
    """Servicio con límite de 1 byte para forzar error de tamaño."""
    return PDFExtractorService(max_file_size_mb=1)

# — Tests de validación —————

class TestValidatePDF:
    def test_validate_pdf_valid_file(self, service, sample_pdf_bytes):
        """Un PDF válido debe retornar True sin lanzar excepción."""
        assert service.validate_pdf(sample_pdf_bytes) is True

    def test_validate_pdf_invalid_file(self, service, invalid_file_bytes):
        """Bytes que no son PDF deben lanzar PDFValidationError."""
        with pytest.raises(PDFValidationError):
            service.validate_pdf(invalid_file_bytes)

    def test_validate_pdf_exceeds_size_limit(self, strict_service, oversized_pdf_bytes):
        """PDF que supera el límite de tamaño debe lanzar PDFValidationError."""
        with pytest.raises(PDFValidationError, match="tamaño máximo"):
            strict_service.validate_pdf(oversized_pdf_bytes)

    def test_validate_pdf_missing_pdf_header(self, service):
        """Bytes sin firma %PDF deben lanzar PDFValidationError."""
        with pytest.raises(PDFValidationError):
            service.validate_pdf(b"PK\x03\x04") # ZIP signature, not PDF

# — Tests de checksum —————

class TestCalculateChecksum:
    def test_calculate_checksum_deterministic(self, service, sample_pdf_bytes):
        """El mismo archivo debe producir siempre el mismo checksum."""
        checksum1 = service.calculate_checksum(sample_pdf_bytes)
        checksum2 = service.calculate_checksum(sample_pdf_bytes)
        assert checksum1 == checksum2

    def test_calculate_checksum_different_files(
        self, service, sample_pdf_bytes, empty_pdf_bytes
    ):

```

```

    """Archivos distintos deben producir checksums distintos."""
    checksum1 = service.calculate_checksum(sample_pdf_bytes)
    checksum2 = service.calculate_checksum(empty_pdf_bytes)
    assert checksum1 != checksum2

def test_calculate_checksum_is_sha256(self, service, sample_pdf_bytes):
    """El checksum debe ser un hash SHA-256 (64 caracteres hexadecimales)."""
    checksum = service.calculate_checksum(sample_pdf_bytes)
    assert len(checksum) == 64
    assert all(c in "0123456789abcdef" for c in checksum)

# — Tests de extracción de texto —————

class TestExtractText:
    def test_extract_text_from_valid_pdf(self, service, sample_pdf_bytes):
        """Un PDF válido debe retornar un string (puede estar vacío si no tiene texto)."""
        text = service.extract_text(sample_pdf_bytes)
        assert isinstance(text, str)

    def test_extract_text_empty_pdf(self, service, empty_pdf_bytes):
        """Un PDF sin texto seleccionable debe retornar string vacío."""
        text = service.extract_text(empty_pdf_bytes)
        assert text == ""

    def test_extract_text_returns_string_type(self, service, sample_pdf_bytes):
        """extract_text siempre debe retornar str, nunca None."""
        result = service.extract_text(sample_pdf_bytes)
        assert result is not None
        assert isinstance(result, str)

# — Tests de extracción de metadata —————

class TestExtractMetadata:
    def test_extract_metadata_returns_dict(self, service, sample_pdf_bytes):
        """extract_metadata debe retornar un dict."""
        metadata = service.extract_metadata(sample_pdf_bytes)
        assert isinstance(metadata, dict)

    def test_extract_metadata_keys_are_lowercase(self, service, sample_pdf_bytes):
        """Las claves del dict de metadata deben estar en minúsculas sin '/'. """
        metadata = service.extract_metadata(sample_pdf_bytes)
        for key in metadata:
            assert key == key.lower(), f"Clave '{key}' no está en minúsculas"
            assert not key.startswith("/"), f"Clave '{key}' no debe comenzar con '/'"

    def test_extract_metadata_author(self, service, sample_pdf_bytes):
        """El PDF de prueba debe contener el campo 'author'. """
        metadata = service.extract_metadata(sample_pdf_bytes)
        assert "author" in metadata
        assert metadata["author"] == "Test Author"

    def test_extract_metadata_title(self, service, sample_pdf_bytes):
        """El PDF de prueba debe contener el campo 'title'. """
        metadata = service.extract_metadata(sample_pdf_bytes)
        assert "title" in metadata
        assert metadata["title"] == "Test PDF"

    def test_extract_metadata_empty_pdf(self, service, empty_pdf_bytes):
        """Un PDF sin metadata debe retornar dict vacío o dict con pocos campos. """

```

```

        metadata = service.extract_metadata(empty_pdf_bytes)
        assert isinstance(metadata, dict)

# — Tests del método principal extract() —————

class TestExtract:
    def test_extract_returns_extracted_document(self, service, sample_pdf_bytes):
        """extract() debe retornar una instancia de ExtractedDocument."""
        result = service.extract(sample_pdf_bytes)
        assert isinstance(result, ExtractedDocument)

    def test_extract_document_has_all_fields(self, service, sample_pdf_bytes):
        """El ExtractedDocument retornado debe tener todos los campos requeridos."""
        result = service.extract(sample_pdf_bytes)
        assert hasattr(result, "text")
        assert hasattr(result, "checksum")
        assert hasattr(result, "page_count")
        assert hasattr(result, "metadata")

    def test_extract_page_count_positive(self, service, sample_pdf_bytes):
        """Un PDF válido debe tener page_count >= 1."""
        result = service.extract(sample_pdf_bytes)
        assert result.page_count >= 1

    def test_extract_invalid_pdf_raises(self, service, invalid_file_bytes):
        """extract() debe propagar PDFValidationError para archivos inválidos."""
        with pytest.raises(PDFValidationError):
            service.extract(invalid_file_bytes)

```

[NEW] tests/unit/extractor/__init__.py

```
# tests/unit/extractor package
```

Cómo ejecutar los tests

```
# Instalar dependencias
uv pip install -r requirements.txt

# Ejecutar solo los tests del extractor
pytest tests/unit/extractor/ -v

# Con reporte de cobertura
pytest tests/unit/extractor/ -v --cov=app/services/extractor --cov-report=term-missing
```

Criterios de aceptación

- [] Los tests se ejecutan con `pytest tests/unit/extractor/ -v` sin errores de importación
- [] Cobertura $\geq 90\%$ de `app/services/extractor/pdf_extractor.py`
- [] Todos los tests pasan con la implementación del sub-issue 1B
- [] Los fixtures de `conftest.py` generan PDFs en memoria (sin archivos temporales)
- [] Cada clase de test cubre exactamente un método del servicio
- [] Los tests de error validan el tipo de excepción y el mensaje

Resumen de distribución

Sub-issue	Responsable	Archivos principales	Depende de	Duración
1A Config & Models	Integrante A	models.py, settings.py, e	—	~1–2h
1B PDFExtractorService	Integrante B	pdf_extractor.py, __init__.	1A	~2–3h
1C Tests TDD	Integrante C	conftest.py, test_pdf_extr	1A (1B para pasar)	~2–3h

Orden de merge

```
1A → (1B || 1C) → PR unificado → Bloque 2 + Bloque 3
```

>> TIP

El integrante de 1C puede comenzar a escribir los tests en paralelo con 1B, usando `pytest.mark.skip` en los tests que aún no puedan ejecutarse. Cuando 1B esté listo, solo hay que quitar los skips.